FUNDATEC

Procesamiento Digital de Señales

Profesor:
Michael Grüner

Estudiante:
Noel Pérez Cáceres

III Cuatrimestre 2026

Tarea 4

Solución Tarea 4

Sea un sistema digital caracterizado por la función de transferencia:

$$H(z) = \frac{1-\alpha}{1-\alpha z^{-k}}$$

1. Grafique el diagrama de polos y ceros de dicho filtro para $\alpha = 0.8$ y $k = 10$.

Multiplicando numerador y denominador por z^k:

$$H(z) = (1-\alpha)\, z^k / (z^k - \alpha)$$

- *Ceros:* el numerador es z^k, por lo que hay k ceros en $z = 0$ (multiplicidad k).
- *Polos:* $z^k = \alpha$, es decir $p_m = \alpha^{1/k}\, e^{j2\pi m/k}$, $m = 0, \ldots, k-1$: k polos equiespaciados sobre un círculo de radio $\alpha^{1/k}$.

Para $\alpha = 0.8$ y $k = 10$ el radio es $0.8^{0.1} \approx 0.9779$. Como todos los polos están dentro del círculo unitario (y para toda $\alpha < 1$) el sistema es estable. Además, al estar los polos cerca del círculo unitario en 10 ángulos equiespaciados, la respuesta en frecuencia presenta 10 picos (estructura de peine) por período.

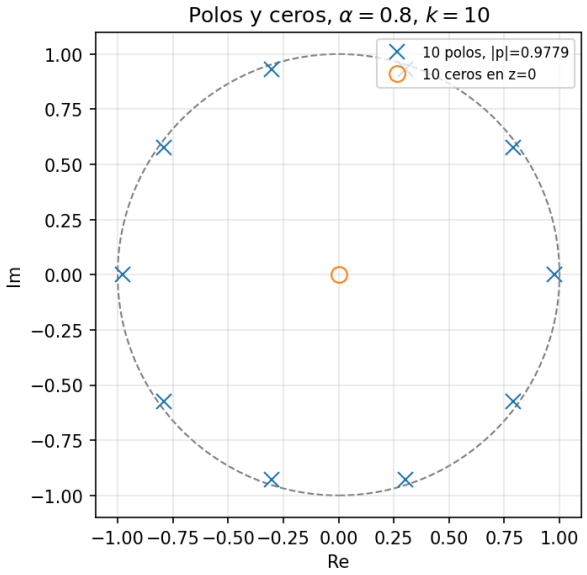


Figura 1. Diagrama de polos y ceros para α = 0.8, k = 10.

2. Encuentre la ecuación de diferencias equivalente para dicho filtro (para todo α y k).

Con $Y(z)/X(z) = H(z)$:

$$(1 - \alpha\, z^{-k})\; Y(z) = (1 - \alpha)\; X(z)$$

$$Y(z) - \alpha\, z^{-k} Y(z) = (1 - \alpha)\; X(z)$$

Aplicando la transformada Z inversa y la propiedad de desplazamiento temporal (z^−k retardo de k muestras), para todo α y k:

$$y[n] = \alpha\, y[n - k] + (1 - \alpha)\; x[n]$$

Es un filtro recursivo causal: cada salida es la entrada escalada por $(1 - \alpha)$ más la salida de hace k muestras atenuada por α. La ganancia en continua es $H(1) = 1$.

3. Implemente dicho filtro en Python para los valores:

k	α
$F_s \times 50$ ms	0.6
$F_s \times 250$ ms	0.4
$F_s \times 500$ ms	0.2

y compruebe su implementación con una señal de audio muestreada a la frecuencia de muestreo F_s. Para ello utilice la función scipy.signal.lfilter (o un equivalente). **No utilice la convolución directamente**.

Los parámetros dependen de la frecuencia de muestreo: $k = F_s \times$ tiempo. La señal de prueba (`test.wav`) tiene $F_s = 44100$ Hz, de modo que:

Retardo	k (muestras)	α	Tipo
50 ms	2205	0.6	Reverberación corta
250 ms	11025	0.4	Eco audible
500 ms	22050	0.2	Eco largo

Se usa `lfilter` con $b = [1 - \alpha]$ y $a = [1, 0, \ldots, 0, -\alpha]$ (longitud $k + 1$, con $-\alpha$ en la posición k), sin recurrir a la convolución directa:

```
def coefs(a, k):
    b = np.array([1 - a])
    den = np.zeros(k + 1); den[0] = 1; den[k] = -a
    return b, den


for ms, al in [(50, .6), (250, .4), (500, .2)]:
    k = int(round(Fs * ms / 1000))
```

```
    b, den = coefs(al, k)
    y = lfilter(b, den, x)
```

La Figura 2 muestra el resultado para los tres casos: la salida es la señal original mezclada con réplicas atenuadas cada k muestras, y la atenuación de cada réplica sucesiva es α, por lo que el eco decae más rápido a medida que α es menor.

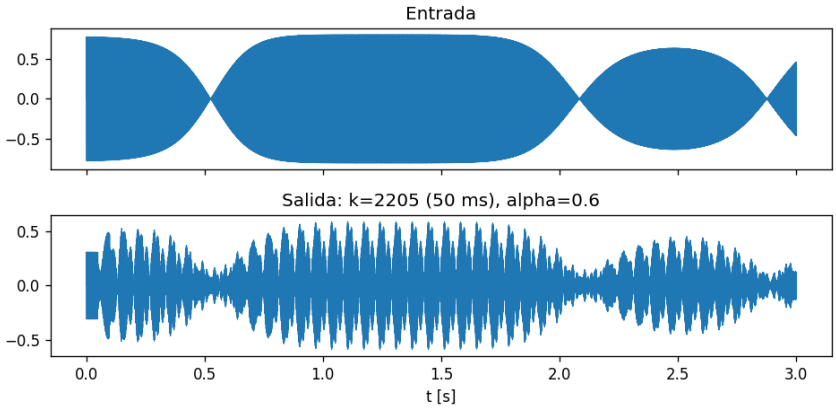


Figura 2a. Entrada y salida, k = 2205 (50 ms), α = 0.6.

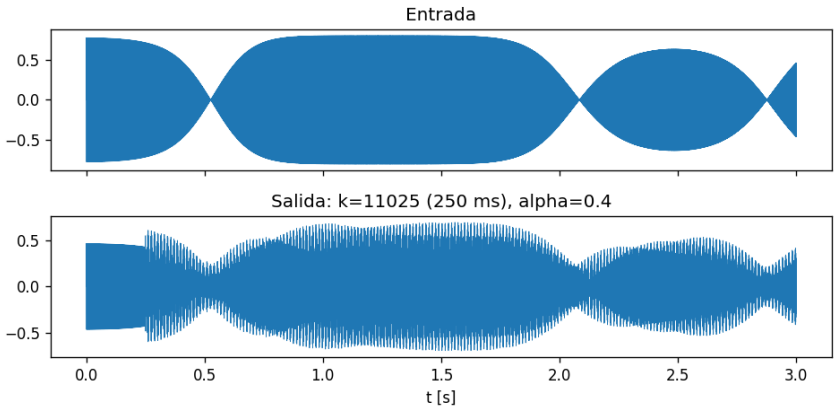


Figura 2b. Entrada y salida, k = 11025 (250 ms), α = 0.4.

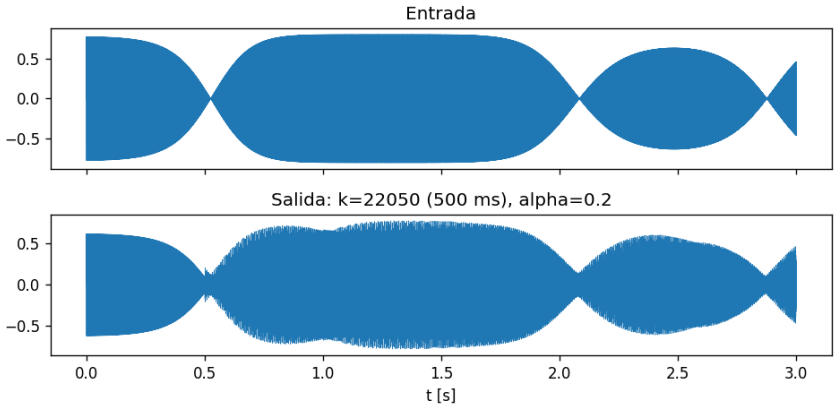


Figura 2c. Entrada y salida, k = 22050 (500 ms), α = 0.2.

4. Implemente su filtro en C. Para ello utilice el programa de prueba entregado con esta tarea, que debe ser compilado y ejecutado en GNU/Linux. Debe modificar únicamente el archivo tarea04.c (y eventualmente tarea04.h). El programa está realizado para capturar datos del micrófono si no se entrega ningún argumento de entrada, o del archivo recibido a través del *pipe*; y pasar a la línea de salida de la tarjeta de sonido los búferes procesados. Todo el proceso de captura y ejecución utiliza jack: servidor de audio basado en ALSA (*Advanced Linux Sound Architecture*).

   Junto al código encontrará un *README.md* con instrucciones sobre cómo ejecutar la aplicación.

Se modificó únicamente `tarea04.c`. Se emplea un buffer circular de k muestras que almacena las últimas k salidas: en cada muestra se lee $y[n-k]$ de la posición actual, se calcula $y[n]$, se sobreescribe esa posición con el nuevo valor y se avanza el índice módulo k. De esta forma el costo es constante por muestra (una multiplicación suma) y el estado persiste entre llamadas a `process()` sin importar el tamaño del buffer de Jack. `init()` reserva el buffer (con `calloc`) a partir de F_s. La variable de entorno `PRESET` (0, 1 o 2) selecciona el par (k, α) de la tabla anterior.

```
#include <string.h>
#include <stdio.h>
#include <stdlib.h>


/*
 * Filtro H(z) = (1-a) / (1 - a z^-k)
 * Ecuacion de diferencias: y[n] = a*y[n-k] + (1-a)*x[n]
 *
 * Se usa un buffer circular de k muestras con las ultimas salidas.
 * Preset seleccionable con la variable de entorno PRESET (0, 1 o 2):
 *   0: k = Fs*50 ms,  a = 0.6   (por defecto)
 *   1: k = Fs*250 ms, a = 0.4
 *   2: k = Fs*500 ms, a = 0.2
 * Ejemplo: PRESET=2 ./jack < test.wav
 */
static const float  alphas[3] = {0.6f, 0.4f, 0.2f};
```

```
static const float  delays[3] = {0.050f, 0.250f, 0.500f}; /* segundos */

static float*       ybuf  = NULL; /* ultimas k salidas */
static unsigned int K     = 1;    /* retardo en muestras */
static unsigned int pos   = 0;    /* posicion actual en el buffer circular */
static float        alpha = 0.6f;

void init(const unsigned int Fs) {
  int p = 0;
  const char* env = getenv("PRESET");
  if (env != NULL) p = atoi(env);
  if (p < 0 || p > 2) p = 0;

  alpha = alphas[p];
  K     = (unsigned int)(Fs * delays[p] + 0.5f);
  if (K < 1) K = 1;

  free(ybuf);
  ybuf = (float*)calloc(K, sizeof(float));
  pos  = 0;
  fprintf(stderr, "Preset %d: k=%u muestras, alpha=%.2f\n", p, K, alpha);
}

/**
 * Process a buffer of data (mono).
 *
 * @return zero if everything is ok (valor esperado por jack).
 */
int process(const unsigned int Fs,
            const int nframes,
            const float* in,
            float* out) {
  const float g = 1.0f - alpha;
  int n;
  if (ybuf == NULL) { /* sin init: pasar la senal */
    memcpy(out, in, sizeof(float)*nframes);
    return 0;
  }
  for (n = 0; n < nframes; ++n) {
    const float y = alpha*ybuf[pos] + g*in[n]; /* ybuf[pos] = y[n-k] */
    ybuf[pos] = y;
    out[n]    = y;
    if (++pos == K) pos = 0;
  }
  return 0; /* everything is ok */
}
```

Se compiló `tarea04.c` con un programa de prueba que procesa `test.wav` en bloques de 1024 muestras y se comparó la salida contra `lfilter` sobre las mismas muestras. El error absoluto máximo fue de aproximadamente 8×10^{-8} para los tres *presets*, consistente con la precisión de punto flotante de 32 bits. Por tanto, las implementaciones en C y Python son equivalentes.